JWT Authentication — Complete Guide

Mechanism, architecture, happy & negative paths, and a full hands-on Spring Boot implementation.

Contents

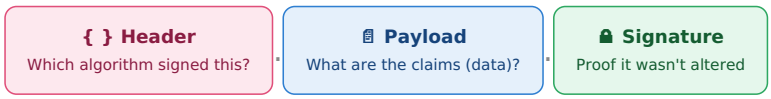
1. What is a JWT, and why 3 parts?
2. Encoding vs. encryption — a common misconception
3. System architecture & end-to-end flow
4. Happy path — step by step
5. Negative path — tampering, expiry, algorithm attacks
6. Security best practices
7. Full Spring Boot implementation (all files)
8. Running & testing the demo
9. Quick reference

1. What is a JWT, and why 3 parts? Beginner

A JSON Web Token (JWT) is a compact, URL-safe string used to represent claims (facts) about a user, transmitted between two parties. It looks like this:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIn0.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

It's three base64url-encoded segments joined by dots — `header.payload.signature` — each with a distinct job:



They're kept separate (rather than one blob) because each is decoded and used differently: the header tells the verifier *how* to check the signature before it even looks at the payload; the payload carries the actual data; the signature is a cryptographic value, not human-readable JSON. Splitting them lets libraries parse and validate efficiently without ambiguity.

Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

`alg` is the signing algorithm (HS256 = HMAC-SHA256, a shared-secret scheme; RS256 = RSA, a public/private key scheme). `typ` just marks it as a JWT.

Payload (Claims)

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true,
  "iat": 1516239022
}
```

Common standard claims: `sub` (subject/user id), `iat` (issued-at), `exp` (expiry), `iss` (issuer). You can add custom claims like `role` or `admin`.

Signature

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secretKey
)
```

2. Encoding vs. Encryption Beginner

> **Common misconception:** a JWT is NOT encrypted. Header and payload are only base64url-*encoded* — anyone can decode and read them (try pasting a token into jwt.io — no secret needed to view the claims). Never put passwords, credit card numbers, or other secrets in the payload.

What the signature *does* guarantee is **integrity** — if a single character in the header or payload changes, the signature will no longer match. That's

the whole security model: not secrecy, but tamper-evidence.

3. System Architecture & End-to-End Flow Intermediate

1 Login
Client sends username/password to `POST /login`.

2 Verify credentials
Server checks against the user database.

3 Issue token
Server signs header+payload with its secret key → returns JWT.

↓

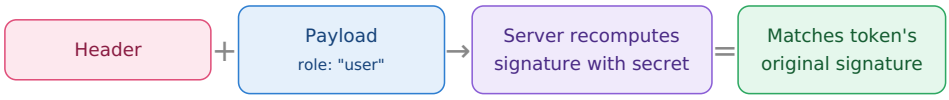
4 Store token
Client keeps it (memory or httpOnly cookie — never localStorage for sensitive apps).

5 Authenticated request
Client sends `Authorization: Bearer <token>` on each call.

6 Server verification
A filter/middleware recomputes the signature and checks `exp` before the request reaches business logic.

Because the server doesn't need to look anything up in a session store to validate the token (the signature check is self-contained), this pattern scales horizontally very well — any server instance holding the same secret can verify any token. That's the main reason JWTs are popular for stateless REST APIs and microservices.

4. Happy Path — Step by Step Beginner

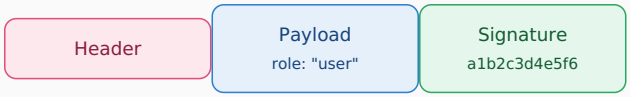


1. Client authenticates once with credentials.
2. Server issues a signed JWT with an expiry.
3. Client attaches the token to every subsequent request.
4. Server recomputes the HMAC over the received header+payload using its secret.
5. Recomputed signature equals the token's signature, and `exp` hasn't passed.

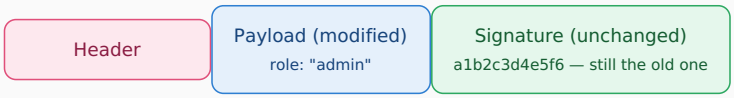
✓ **Verification successful** — request is trusted, `200 OK`, access granted.

5. Negative Path — Tampering, Expiry, Algorithm Attacks Intermediate

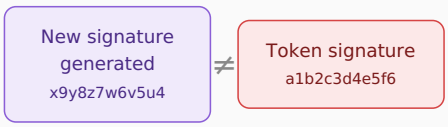
Step 1 — original token issued by server:



Step 2 — attacker modifies the payload (no access to the secret key):



Step 3 — server re-verifies:



✗ **No match → 401 Authentication Failed. Access denied.**

Other negative scenarios a production server must handle

Scenario	Why it's rejected
Payload tampered (role/privilege escalation)	Recomputed signature won't match — covered above.
Expired token (`exp` in the past)	Library throws an expiry exception even if the signature is valid.
"alg: none" attack	Attacker sets header `alg` to `none` and strips the signature, hoping a lax verifier skips the check. **Fix:** always specify the expected algorithm explicitly when verifying; never trust `alg` from the token itself.
Algorithm confusion (RS256 → HS256)	If a server verifies with a public key but accepts HS256, an attacker can sign a fake token using the public key as an HMAC secret (the public key is, well, public). **Fix:** pin the algorithm server-side per key type.
Malformed token (not 3 segments /	

bad base64)	Parser fails before signature check — reject immediately.
Token replay after logout (stateless JWTs can't be revoked individually)	Mitigate with short expiries + refresh tokens, or a server-side denylist for the token's `jti` claim.

6. Security Best Practices Advanced

- Keep access tokens short-lived (minutes–hours); use a separate, longer-lived refresh token to renew them.
- Store tokens in an httpOnly, Secure, SameSite cookie for browser apps — not `localStorage` (XSS-exposed).
- Always set and check `exp`; consider `nbf` (not-before) for delayed-activation tokens.
- Use at least a 256-bit secret for HS256; rotate secrets periodically and support key IDs (`kid`) for smooth rotation.
- Explicitly pin the accepted algorithm(s) when verifying — never read `alg` from the untrusted token to decide how to verify it.
- Don't put sensitive data (passwords, SSNs, tokens) in the payload — it's readable by anyone.
- Use HTTPS everywhere — a JWT sent over plain HTTP can be captured and replayed.

7. Full Spring Boot Implementation Advanced

Below is the complete, runnable project. Dependencies: `spring-boot-starter-web`, `spring-boot-starter-security`, and `jjwt` 0.12.6 for signing/verifying.

pom.xml

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <!-- JJWT: JWT creation & verification -->
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.6</version>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.12.6</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.12.6</version>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

JwtDemoApplication.java

```
package com.example.jwtdemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class JwtDemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(JwtDemoApplication.class, args);
    }
}
```

JwtService.java — sign & verify (the core mechanism)

```java
package com.example.jwtdemo;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.ExpiredJwtException;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import io.jsonwebtoken.security.SignatureException;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.Map;

@Service
public class JwtService {

    // In production: load from a secrets manager / env var, never hard-code.
    private static final String SECRET = "a-string-secret-at-least-256-bits-long!!";
    private final SecretKey key = Keys.hmacShaKeyFor(SECRET.getBytes(StandardCharsets.UTF_8));
    private static final long EXPIRY_MS = 60 * 60 * 1000; // 1 hour

    /** HAPPY PATH — issuance: server signs a new token at login time. */
    public String generateToken(String subject, Map<String, Object> claims) {
        Date now = new Date();
        Date expiry = new Date(now.getTime() + EXPIRY_MS);

        return Jwts.builder()
                .claims(claims)
                .subject(subject)
                .issuedAt(now)
                .expiration(expiry)
                .signWith(key, Jwts.SIG.HS256)
                .compact();
    }

    /** Recomputes the signature and validates claims on every request. */
    public Claims verifyToken(String token) {
        return Jwts.parser()
                .verifyWith(key)
                .build()
                .parseSignedClaims(token)
                .getPayload();
    }

    public record VerificationResult(boolean valid, String message, Claims claims) {}

    /** Convenience wrapper — never throws; used by controller and demo runner. */
    public VerificationResult tryVerify(String token) {
        try {
            Claims claims = verifyToken(token);
            return new VerificationResult(true, "Verification successful. Access granted.", claims);
        } catch (SignatureException e) {
            return new VerificationResult(false, "Invalid signature -> 401 Access denied.", null);
        } catch (ExpiredJwtException e) {
            return new VerificationResult(false, "Token expired -> 401 Access denied.", null);
        } catch (JwtException e) {
            return new VerificationResult(false, "Malformed token -> 401 Access denied.", null);
        }
    }
}
```

AuthController.java — /login and /profile

```
package com.example.jwtdemo;

import io.jsonwebtoken.Claims;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
public class AuthController {

    private final JwtService jwtService;

    public AuthController(JwtService jwtService) {
        this.jwtService = jwtService;
    }

    public record LoginRequest(String username, String password) {}

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest req) {
        if (!"alice".equals(req.username()) || !"password123".equals(req.password())) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("error", "Invalid credentials"));
        }
        String token = jwtService.generateToken(req.username(), Map.of("role", "user"));
        return ResponseEntity.ok(Map.of("token", token));
    }

    @GetMapping("/profile")
    public ResponseEntity<?> profile(@RequestHeader("Authorization") String authHeader) {
        String token = authHeader.replace("Bearer ", "");
        JwtService.VerificationResult result = jwtService.tryVerify(token);

        if (!result.valid()) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of("error", result.message()));
        }
        Claims claims = result.claims();
        return ResponseEntity.ok(Map.of(
                "message", result.message(),
                "subject", claims.getSubject(),
                "role", claims.get("role")
        ));
    }
}
```

JwtAuthFilter.java — enforces verification on every request

```java
package com.example.jwtdemo;

import io.jsonwebtoken.Claims;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.List;

@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    public JwtAuthFilter(JwtService jwtService) {
        this.jwtService = jwtService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain chain)
            throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);
            JwtService.VerificationResult result = jwtService.tryVerify(token);

            if (result.valid()) {
                Claims claims = result.claims();
                var auth = new UsernamePasswordAuthenticationToken(claims.getSubject(), null, List.of());
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
            // If invalid: leave unauthenticated; Spring Security returns 401 for protected routes.
        }
        chain.doFilter(request, response);
    }
}
```

SecurityConfig.java — wires the filter into the chain

```java
package com.example.jwtdemo;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;

    public SecurityConfig(JwtAuthFilter jwtAuthFilter) {
        this.jwtAuthFilter = jwtAuthFilter;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/login").permitAll()
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

JwtDemoRunner.java — console walkthrough of happy & negative paths

```
package com.example.jwtdemo;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.boot.CommandLineRunner;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Date;
import java.util.Map;

@Component
@Order(1)
public class JwtDemoRunner implements CommandLineRunner {

    private final JwtService jwtService;

    public JwtDemoRunner(JwtService jwtService) {
        this.jwtService = jwtService;
    }

    @Override
    public void run(String... args) {
        line("1. HAPPY PATH: Sign + Verify");
        String token = jwtService.generateToken("123", Map.of("role", "user"));
        System.out.println("Issued token: " + token);

        JwtService.VerificationResult result = jwtService.tryVerify(token);
        System.out.println((result.valid() ? "SUCCESS: " : "FAILED: ") + result.message());

        line("2. NEGATIVE PATH: Attacker modifies payload");
        String[] parts = token.split("\\.");
        String payloadJson = "{\"sub\":\"123\",\"role\":\"admin\",\"iat\":" + (new Date().getTime() / 1000) + "}";
        String tamperedPayloadB64 = Base64.getUrlEncoder().withoutPadding()
                .encodeToString(payloadJson.getBytes(StandardCharsets.UTF_8));
        String tamperedToken = parts[0] + "." + tamperedPayloadB64 + "." + parts[2];

        JwtService.VerificationResult tamperedResult = jwtService.tryVerify(tamperedToken);
        System.out.println((tamperedResult.valid() ? "SUCCESS: " : "FAILED: ") + tamperedResult.message());

        line("3. NEGATIVE PATH: Expired token");
        SecretKey key = Keys.hmacShaKeyFor("a-string-secret-at-least-256-bits-long!!".getBytes(StandardCharsets.UTF_8));
        String expiredToken = Jwts.builder()
                .subject("123").claim("role", "user")
                .issuedAt(new Date(System.currentTimeMillis() - 20_000))
                .expiration(new Date(System.currentTimeMillis() - 10_000))
                .signWith(key, Jwts.SIG.HS256)
                .compact();

        JwtService.VerificationResult expiredResult = jwtService.tryVerify(expiredToken);
        System.out.println((expiredResult.valid() ? "SUCCESS: " : "FAILED: ") + expiredResult.message());
    }

    private void line(String title) {
        System.out.println("\n" + "=".repeat(60) + "\n" + title + "\n" + "=".repeat(60));
    }
}
```

8. Running & Testing the Demo Intermediate

Start the app (auto-runs the console walkthrough, then serves REST endpoints):

```
mvn spring-boot:run
```

Test the happy path:

```
curl -X POST localhost:8080/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}'
# -> { "token": "eyJhbGciOi..." }

curl localhost:8080/profile -H "Authorization: Bearer <token>"
# -> { "message": "Verification successful...", "subject": "alice", "role": "user" }
```

Test the negative path (tamper the token, e.g. change one character in the payload segment):

```
curl localhost:8080/profile -H "Authorization: Bearer <tampered-token>"
# -> 401 { "error": "Invalid signature -> 401 Access denied." }
```

9. Quick Reference Beginner

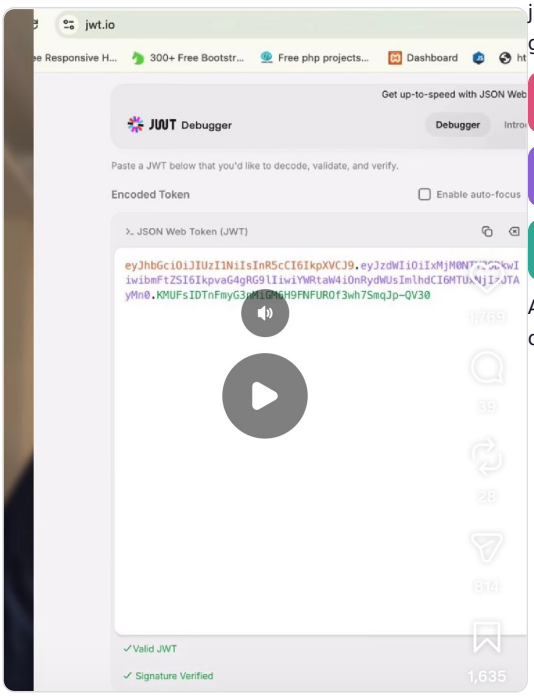
Case	Signature check	Server response
Valid token, not expired	Matches	**200 OK, access granted**
Payload tampered	Does not match	**401 Unauthorized**
`exp` in the past	N/A — rejected on claim check	**401 Token expired**
Malformed / wrong `alg`	N/A	**400 / 401 Invalid token**

Complete JWT guide — structure, architecture, happy/negative paths, security notes, and a full runnable Spring Boot implementation.

jwt.io Walkthrough — How Token Validation Actually Works

Reading the jwt.io debugger screen by screen: every part of the token, its job, and exactly how it drives a valid or invalid result.

1. The Encoded Token — three colored parts

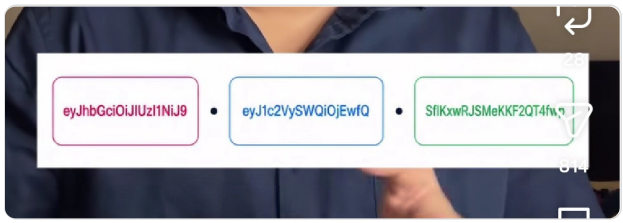


jwt.io — Encoded Token field

jwt.io color-codes the token the moment you paste it in, so you can see the 3 segments at a glance:

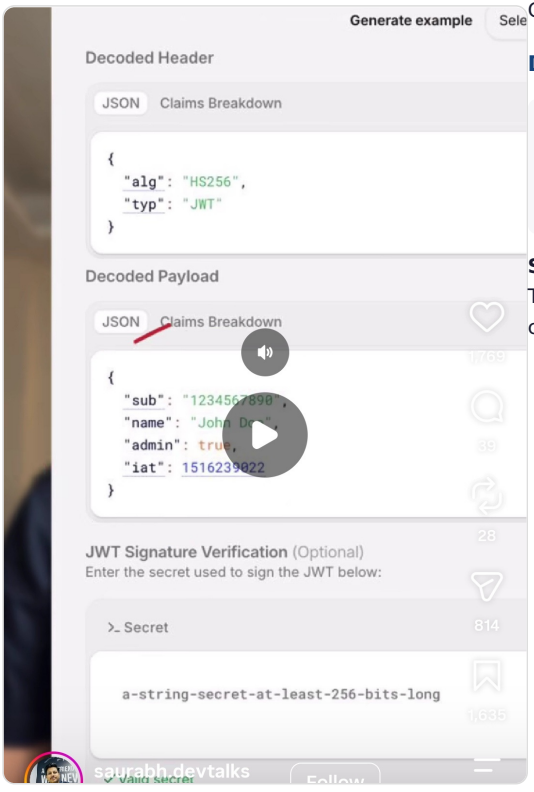
- **Red segment** — the **Header** (base64url JSON)
- **Purple segment** — the **Payload** (base64url JSON, the claims)
- **Teal/blue segment** — the **Signature** (raw HMAC/RSA output, not JSON)

At the bottom it immediately reports ✓ **Valid JWT** and ✓ **Signature Verified** — that's the debugger telling you it recomputed the signature locally and it matched.



The same 3 parts, zoomed in — this is exactly what gets sent in the Authorization header

2. Decoded Header & Payload — what each part is FOR



Clicking on a segment shows its decoded JSON on the right:

Decoded Header

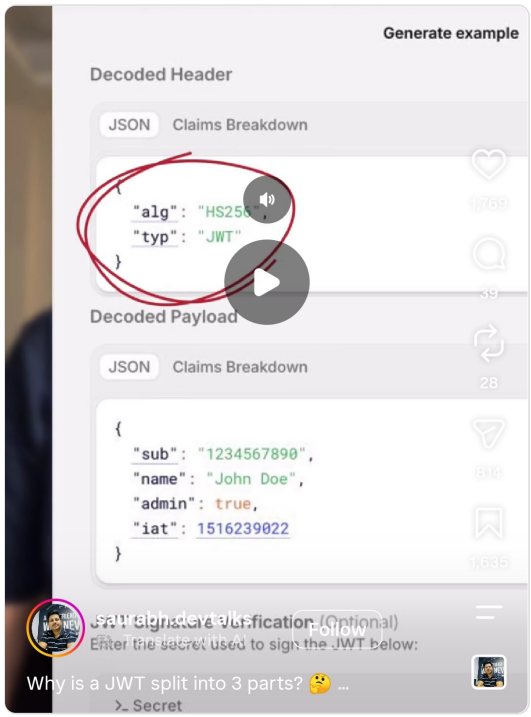
```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Significance: tells the verifier *which algorithm and key type* to use to recompute the signature. This is read *before* verification — it decides the recipe, but the header's own content is still covered by the signature, so it can't be swapped either.

Decoded Payload

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true,
  "iat": 1516239022
}
```

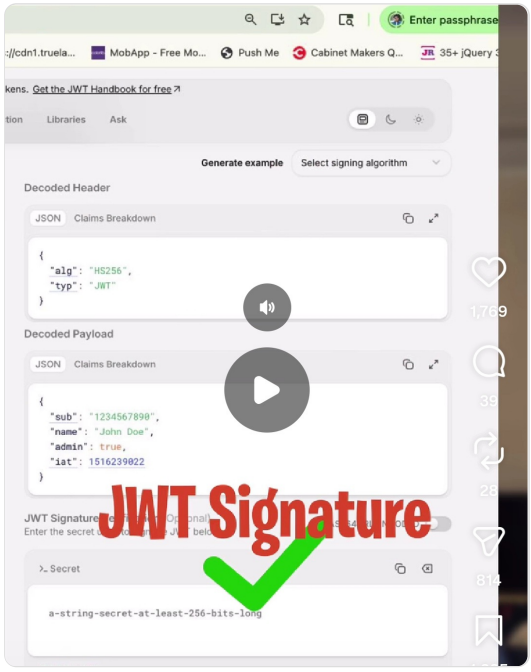
Significance: the actual claims/data about the user — identity (`sub`), custom attributes (`admin`), and timing (`iat` , and where present `exp`). This is the part that matters to your application logic, and it's also the part attackers want to change (e.g. flip `admin` to `true`).



"Why is a JWT split into 3 parts?" — header circled: it's decoded and read independently, before the payload is even considered

This is the question the video poses, and the answer is exactly what we're walking through: each part has a distinct role, and jwt.io's UI structure (separate Header/Payload boxes plus a separate Secret box) mirrors that separation — you decode header and payload independently, but you verify them *together* against the signature.

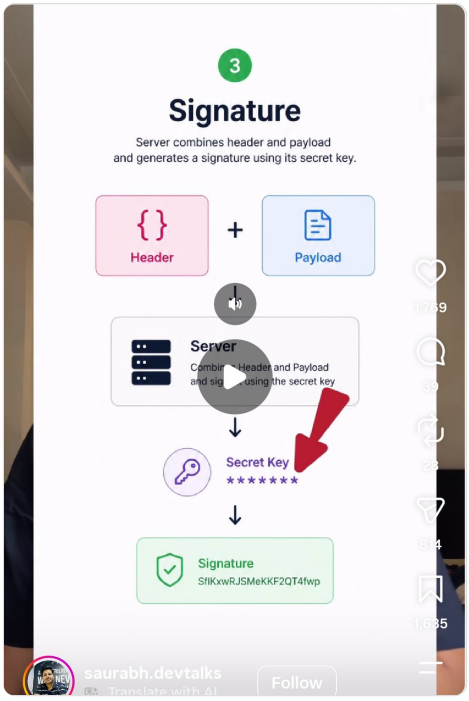
3. The Secret — what makes verification possible at all



"JWT Signature Verification (Optional)" — entering the secret used to sign the token

jwt.io can't verify a signature unless you supply the same secret the server used to sign it (for HS256), or the public key (for RS256). Typing in `a-string-secret-at-least-256-bits-long` lets jwt.io locally recompute the HMAC and compare it to the token's third segment.

What actually happens under the hood (Signature step)



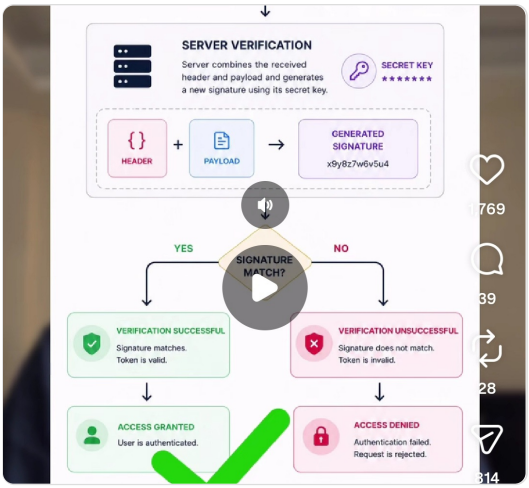
Header + Payload → server signs with its secret key → Signature

The signature is computed as:

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secretKey
)
```

Notice the signature is a function of **both** the header and payload bytes plus the secret. Change either JSON blob by even one character, and the resulting signature is completely different — that's the entire tamper-detection mechanism.

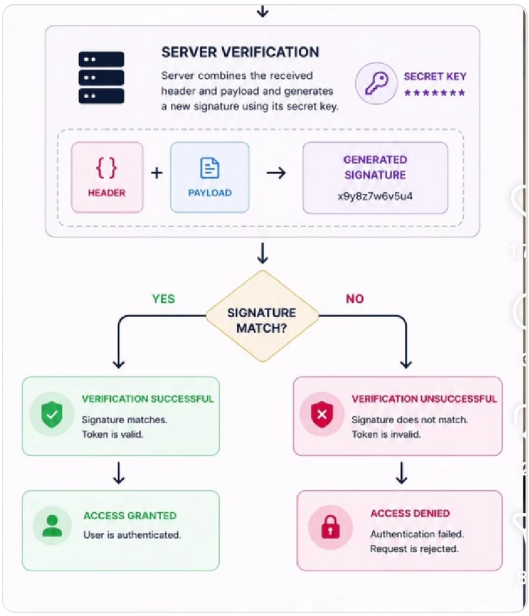
4. Server Verification — the actual go/no-go decision



Server recombines header + payload, signs with its secret key, then compares

On every incoming request, the server:

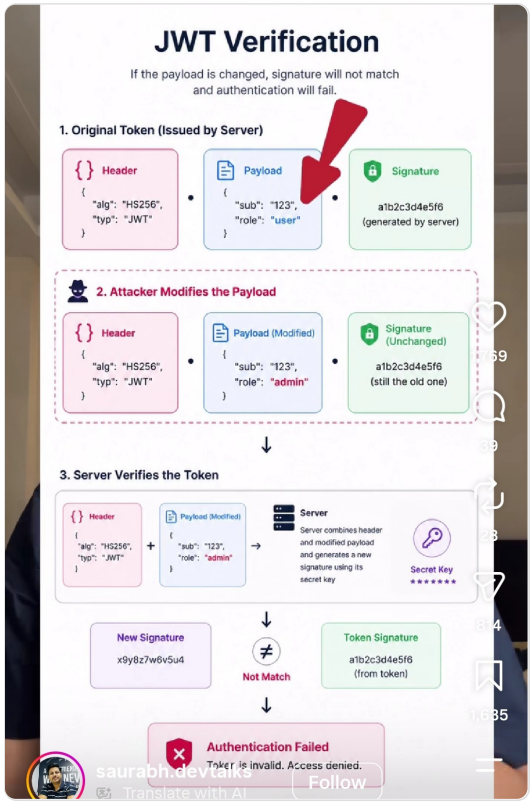
1. Splits the token on `.` into header, payload, signature.
2. Recomputes `HMACSHA256(header + "." + payload, secretKey)` itself — call this the **generated signature**.
3. Compares the generated signature to the signature segment that came with the token.



Signature Match? — the single branching point that decides everything

Branch	Meaning	Result
YES — signatures match	Header+payload haven't changed since signing, and whoever verifies has the correct secret	**Verification Successful → Token is valid → Access Granted**
NO — signatures differ	Either the payload/header was altered after signing, or the wrong secret is being used to verify	**Verification Unsuccessful → Token is invalid → Access Denied**

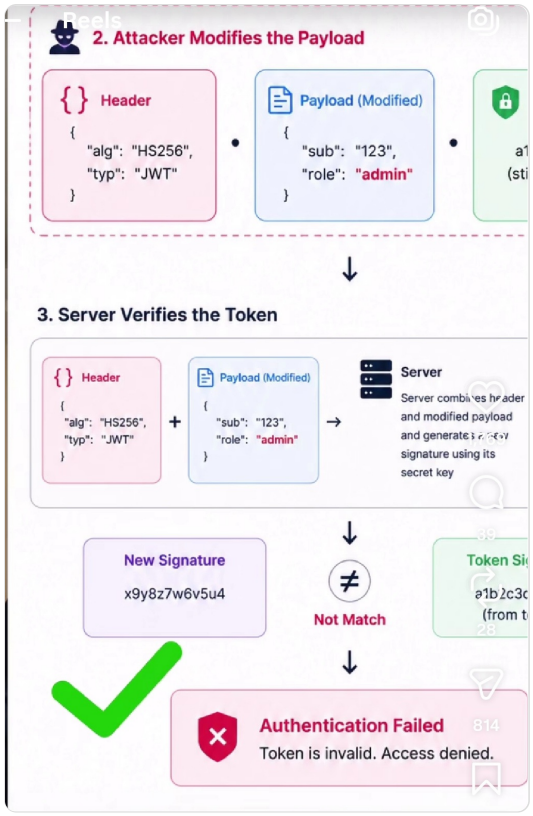
5. Putting it together — a tampered token walked through all 3 parts



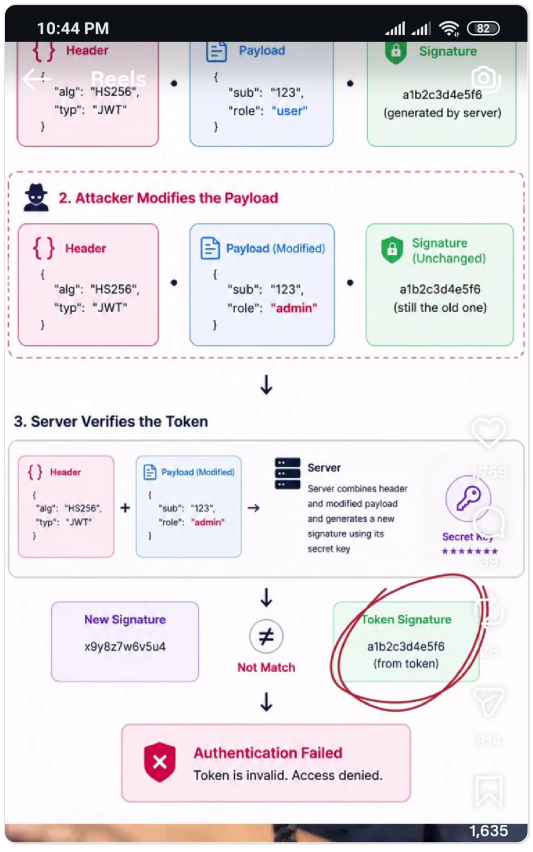
Full tamper trace: original token → attacker edits payload → server re-verifies → mismatch → rejected

Part-by-part role in this trace

Part	Original token	After tampering	Role in the outcome
Header `{"alg":"HS256","typ":"JWT"}`	Unchanged	Unchanged	Still tells the server which algorithm to use to recompute — attacker didn't need to touch this for a payload attack, but it's still covered by the signature, so changing it (e.g. to `alg: none`) would also break verification unless the server naively trusts it.
Payload `{"sub":"123","role":"user"}`	**role: "user"**	**role: "admin" (attacker-edited)**	This is the part attackers modify to escalate privilege. It's freely readable/editable by anyone since it's only base64-encoded — but editing it invalidates the signature relationship.
Signature `a1b2c3d4e5f6`	Generated correctly by server at issuance	Left **unchanged** — attacker has no secret key to make a new valid one	This is the part that *catches* the tampering: it no longer corresponds to the new payload, so recomputation reveals the mismatch.



Server recomputes with the modified payload → gets a brand-new signature (x9y8z7w6v5u4) that doesn't match the token's old one (a1b2c3d4e5f6)



"Not Match" against the original Token Signature → Authentication Failed, Token is invalid, Access denied

6. Summary — how each part participates in validation

Token part	Encoded how	Readable by anyone?	Can be silently altered?	What breaks if altered
Header	base64url(JSON)	Yes	No — covered by signature	Signature mismatch → rejected
Payload	base64url(JSON)	Yes	No — covered by signature	Signature mismatch → rejected
Signature	raw HMAC/RSA bytes, base64url	No (not meaningful JSON, just a hash/signature value)	Attacker can't forge a new one without the secret/private key	N/A — this *is* the check

The elegant part of the design: header and payload are *transparent* (readable, not secret) but *tamper-evident* together, because the signature is a cryptographic function of both, keyed by a secret only the server holds. jwt.io's debugger simply performs the server's exact recomputation client-side, using whatever secret you type in, and shows you the same Match / Not Match outcome the server would produce.

---

jwt.io debugger walkthrough — token anatomy, decoding, signature recomputation, and the tampering trace, illustrated with the actual debugger screens.